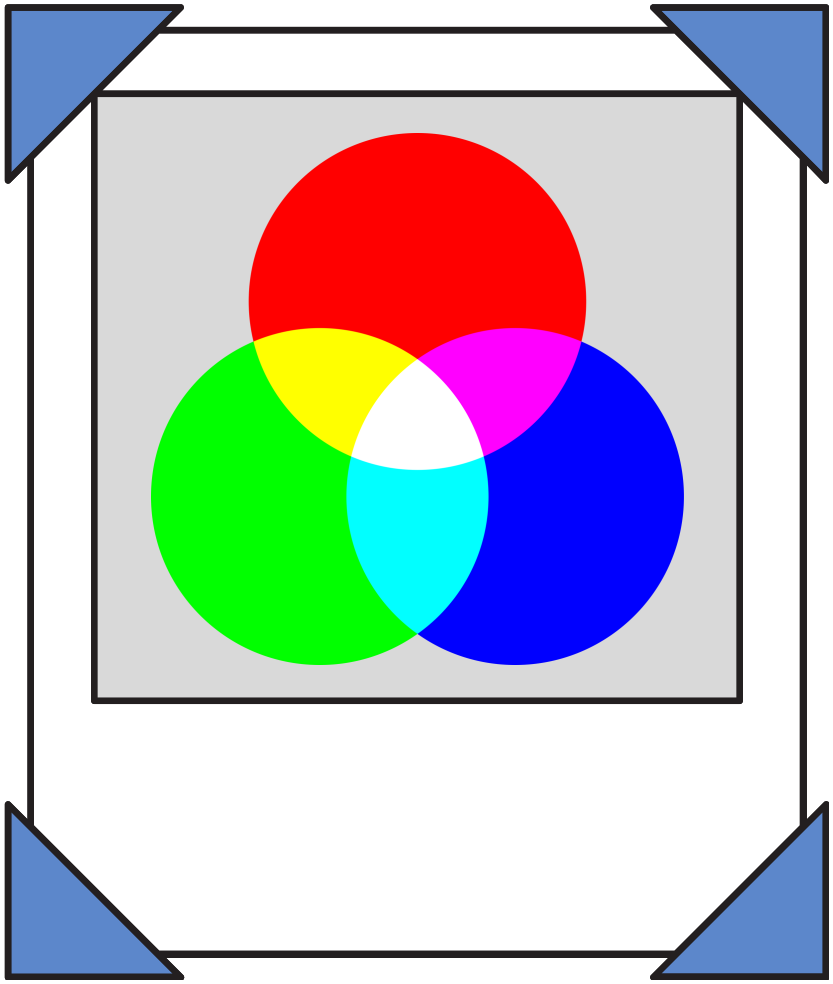
SAE 3.2 FA - PRIMITIVE IMAGE FORMAT
2025 - 2026



Rayan BISSON, Michel TRAN &
Emmanuel SRIVASTAVA-TIAMZON

SOMMAIRE

I	Introduction
III	Le convertisseur
IV	Le visualisateur
V	Conclusion

I INTRODUCTION

Notre groupe est composé de Rayan BISSON, Michel TRAN, Emmanuel SRIVASTAVA-TIAMZON. Le projet Primitive Image Format (PIF), consiste à développer deux programmes, un visualisateur et un convertisseur en Java, dans le but de concevoir et d'exploiter le format d'image compressé sans perte. Ce format vise à résoudre les problèmes de stockage liés aux formats d'images simplifiés en s'appuyant sur des techniques d'encodage optimisées.
L'architecture de notre projet est donc répartie sur deux axes principaux :

1. Le convertisseur :

Ce programme compresse une image en analysant les fréquences de ses canaux de couleurs RVB. Il utilise l'arbre de Huffman puis une conversion en codes canoniques pour attribuer des codes courts aux couleurs fréquentes, minimisant ainsi l'en-tête du fichier **.pif**. Son interface graphique permet de prévisualiser l'image redimensionnée et les tables de calcul avant l'exportation.

2. Le Visualisateur

Ce programme décode les fichiers **.pif** pour reconstituer l'image pixel par pixel. Son interface ergonomique s'adapte automatiquement aux dimensions de l'image (centrée si petite, navigable par **"drag and drop"** sans déborder de l'écran.

L'ensemble du projet respecte un cahier des charges strict : utilisation exclusive de l'API standard Java, versioning sur Gitea, et application rigoureuse de l'architecture MVC (Modèle-Vue-Contrôleur).

CONVERTISSEUR

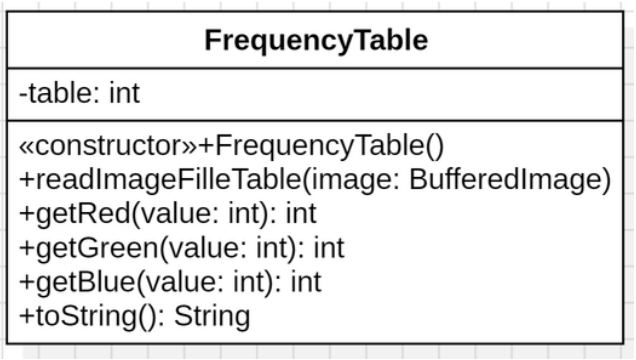
Explication des structures de données par fonctionnalités selon l'ordre d'exécution. (Partie expliquée par Rayan ezezezez)

Les trois tables de fréquence

Pour les tables de fréquences, comme nous devons stocker des valeurs allant de 0 à 255, on se simplifie énormément la tâche en créant un tableau à deux dimensions qui va stocker toutes les valeurs des trois tables de fréquence.

Le fonctionnement des tables de fréquences est assez simple :

- On prends une image en entrée,
- On parcourt tous ses bits 1 par 1,
- On extrait les valeurs de R, G, et B,
- Pour chaque valeur, on incrémente sa fréquence dans son tableau respectif.



Le codage de Huffman

L'arbre de Huffman est implémenté par la classe HuffmanTree, qui contient une attribut qui est une référence vers un objet Node représentant la racine. Chaque Node constitue un élément de l'arbre binaire et possède une fréquence ainsi que deux références vers ses fils gauche et droit. Les feuilles contiennent une valeur correspondant à une intensité RGB, tandis que les nœuds internes contiennent uniquement une fréquence qui est la somme de leurs enfants.
La construction de l'arbre se fait avec une PriorityQueue, pour bien garder l'ordre nécessaire pour mettre en place le codage de Huffman.

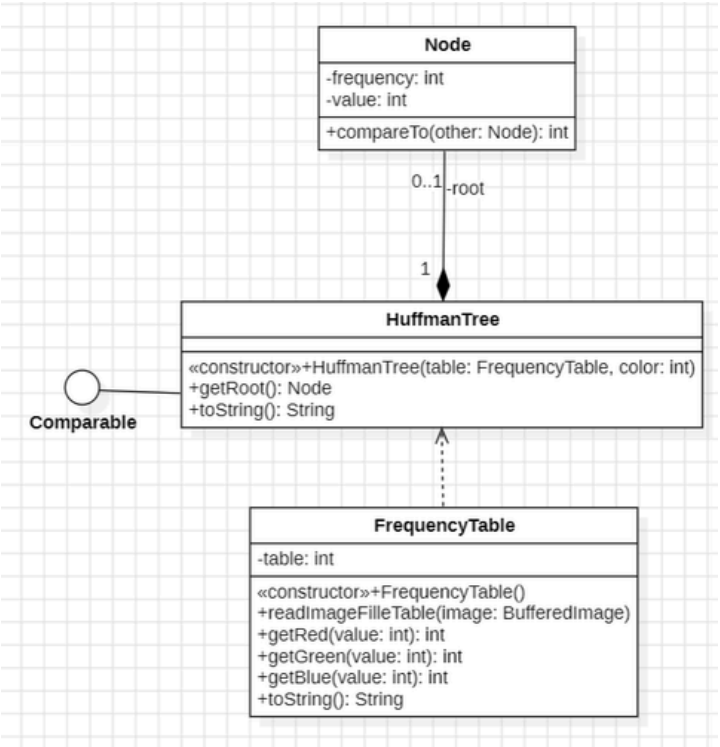
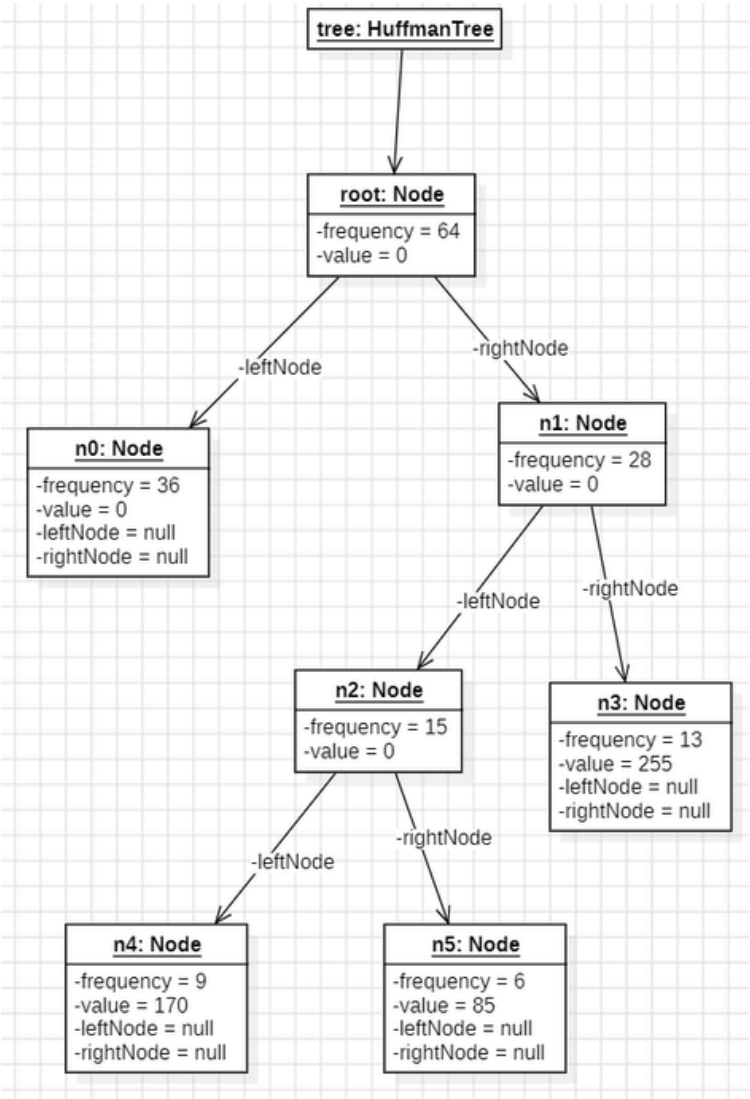


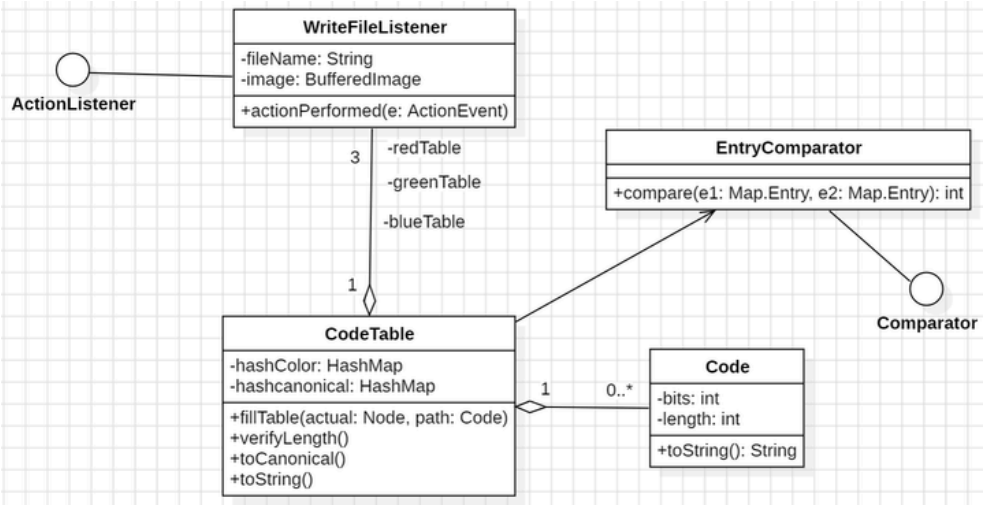
Diagramme d’objet de l'arbre binaire

Ce diagramme montre l'état en mémoire de l'arbre binaire une fois sa construction achevée, en s'appuyant sur l'exemple d'une image de 8x8 pixels. Au sommet se trouve l'objet racine (root), dont l'attribut fréquence (64) correspond au nombre total de pixels analysés. En descendant, on observe les nœuds internes : ce sont des objets créés dynamiquement par l'algorithme pour additionner les fréquences de leurs deux enfants (leurs liaisons leftNode et rightNode). Enfin, aux extrémités se trouvent les feuilles. Ce sont les seuls objets qui ne possèdent pas d'enfants (leurs pointeurs sont null) et qui stockent les véritables valeurs des couleurs (ex: 85, 170, 255). C'est la structure exacte de ces liaisons d'objets que notre méthode récursive va ensuite parcourir pour déterminer le code binaire (0 ou 1) de chaque couleur.



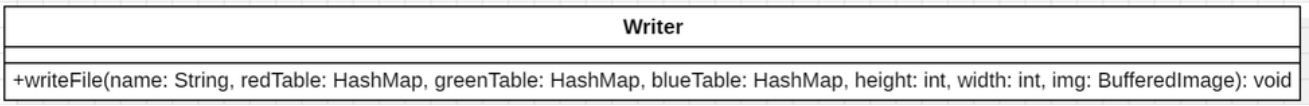
Le code canonique

La table des codes adopte un dictionnaire associative symbole → code binaire, fait par une HashMap.
Les codes sont d'abord générés par parcours récursif de l'arbre de Huffman, puis transformés en codes canoniques par tri selon la longueur du code et par valeur par la suite avec le EntryComparator.



Chaque symbole est directement converti en sa représentation binaire sans navigation dans l'arbre, garantissant

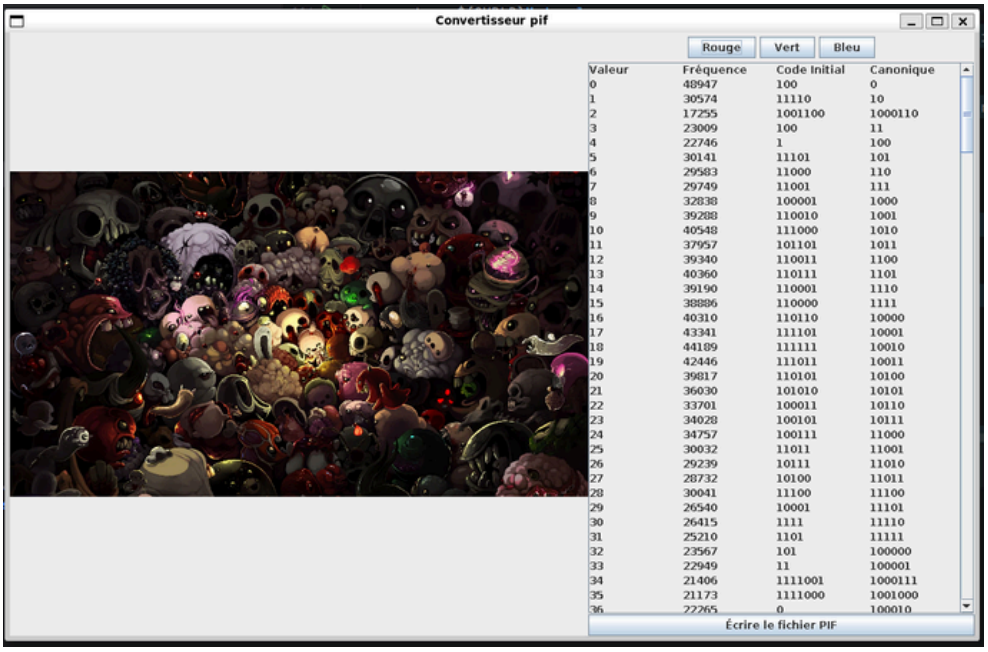
L'écriture du fichier



La classe Writer assure l'écriture complète du fichier compressé .pif.
Le fichier contient successivement les dimensions de l'image, les longueurs des codes canoniques pour chaque composante RGB, puis les données encodées.
L'encodage consiste à remplacer chaque valeur RGB par son code canonique et à concaténer les bits dans un flux continu. Les bits sont regroupés en octets avant écriture, garantissant un stockage compact.
Grâce à l'utilisation de codes canoniques, seules les longueurs sont nécessaires dans le header, ce qui permet au décodeur de reconstruire les codes sans ambiguïté.

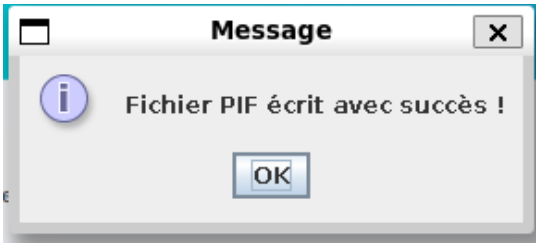
Présentation de l’exécution du convertisseur

Tout d’abord pour lancer le code on utilise les commandes make et les raccourcis que l’on a préparé dans le Makefile. Au démarrage, si aucun fichier n'est passé en argument de ligne de commande, une boîte de dialogue (JFileChooser) s'ouvre pour inviter l'utilisateur à sélectionner une image source. Une fois le fichier chargé, l'interface graphique principale s'affiche. Elle présente un aperçu de l'image (redimensionnée dynamiquement si nécessaire) ainsi qu'un espace interactif composé de trois onglets. Ces derniers permettent d'explorer en détail les données de chaque canal de couleur (Rouge, Vert, Bleu), en affichant sous forme de grille, les valeurs des intensités lumineuses, leurs fréquences d'apparition, les codes de Huffman initiaux générés par l'arbre, ainsi que les codes canoniques finaux.

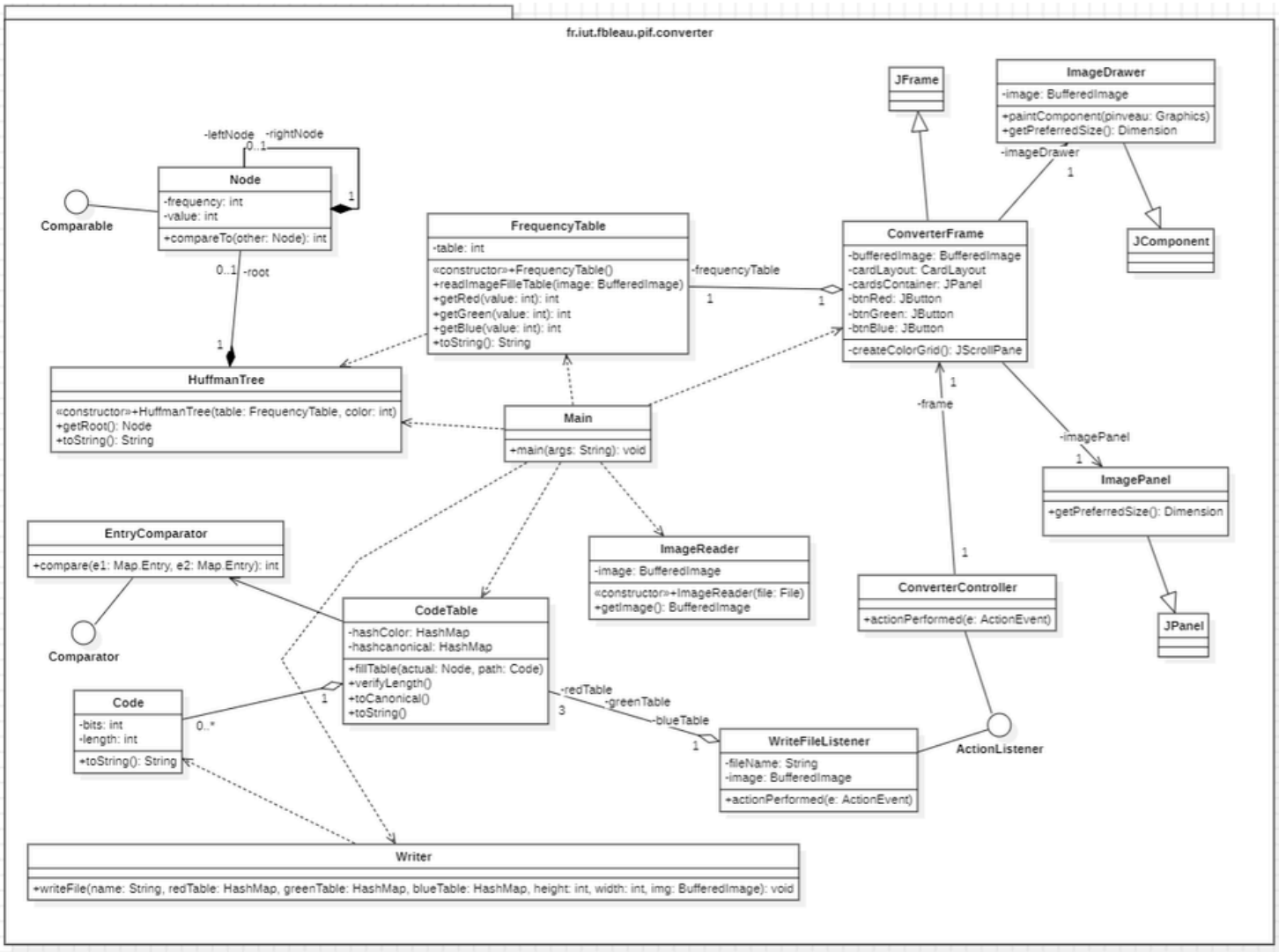


Valeur	Fréquence	Code Initial	Canonique
0	48947	100	0
1	30574	11110	10
2	17255	1001100	1000110
3	23009	100	11
4	22746	1	100
5	30141	11101	101
6	29583	11000	110
7	29749	11001	111
8	32838	100001	1000
9	39288	110010	1001
10	40548	111000	1010
11	37957	101101	1011
12	39340	110011	1100
13	40360	110111	1101
14	39190	110001	1110
15	38886	110000	1111
16	40310	110110	10000
17	43341	111101	10001
18	44189	111111	10010
19	42446	111011	10011
20	39817	110101	10100
21	36030	101010	10101
22	33701	100011	10110
23	34028	100101	10111
24	34757	100111	11000
25	30032	11011	11001
26	29239	10111	11010
27	28732	10100	11011
28	30041	11100	11100
29	26540	10001	11101
30	26415	1111	11110
31	25210	1101	11111
32	23567	101	100000
33	22949	11	100001
34	21406	1111001	1000111
35	21173	1111000	1001000
36	22265	0	100010

Après avoir cliquer le bouton “écrire le fichier PIF” cette fenêtre s’affiche pour confirmer à l’utilisateur que le fichier **.pif** a été généré.



Présentation de la structure du programme et diagramme de classes du convertisseur



Le programme du convertisseur se décompose en 3 grandes parties :

- La première partie contient les classes “FrequencyTable”, “HuffmanTree”, “Node”, “CodeTable”, “Code” et “EntryComparator”. Cette partie s’occupe du cœur algorithmique : le calcul des fréquences d'apparition des couleurs, la construction des arbres binaires et la génération des tables de codes canoniques.
- La deuxième partie contient les classes “ConverterFrame”, “ImagePanel”, “ImageDrawer” et “ImageReader”. Elle s’occupe de charger l'image et de créer l'interface graphique pour l'afficher (en la redimensionnant dynamiquement) tout en présentant les différentes tables de données à l'utilisateur.
- La troisième partie contient les classes “ConverterController”, “WriteFileListener” et “Writer”. Elle s’occupe de gérer les interactions de l’utilisateur (la navigation entre les onglets et le clic sur les boutons) et de procéder à l'écriture finale du fichier binaire .pif avec son en-tête et ses pixels compressés.

L’ensemble de ces 3 parties (qui sont reliées par la classe “Main”) permettent au convertisseur de fonctionner correctement.

Les tables de codes

Tout d'abord on peut décomposer le fichier pif en 3 parties :

- la première contient les dimensions de l'image
- la deuxième contient la longueur de chaque intensité, allant de 0 à 255, pour les tables R, G et B.
- la troisième contient les codes canoniques de chaque composantes.

Pour créer les tables de code on utilisera la partie 2 du fichier. Dans le programme les tables de code sont représentés à travers trois "HashMap", un par composante RGB. Etant donné le flux de données massif, nous avons choisi cette structure de données, pour éviter au programme d'avoir a parcourir un arbre entièrement a chaque valeur décodée. Ainsi l'utilisation de "HashMap" permet de raccourcir le temps décodage de l'application.

Comment ces tables sont elles créée ?

La création des tables se fait via deux méthodes de la classe "FileTreatment", les méthodes "canonicalCodes" et "finalData".

La première méthode permet de générer les codes binaires à partir d'un liste d'objet "DataColor" déjà triée. La création des codes est faites de manière a ce que aucun code ne possède le même début grâce a des décalages binaires. Apres cela la méthode comble le vide avec des 0 pour correspondre a la longueur obtenu dans l'objet "DataColor".

La deuxième méthode va utiliser la première méthode, puis ajouter chaque élément dans le bon "HashMap" correspondant.

L’utilisation de ces tables, le décodage.

Pour cette étape on va utiliser la partie 3 du fichier. Le décodage se fait via l’utilisation de trois méthodes de la classe “CreateImage”, les méthodes “getBit”, “decodeValue” et “buildImage”.

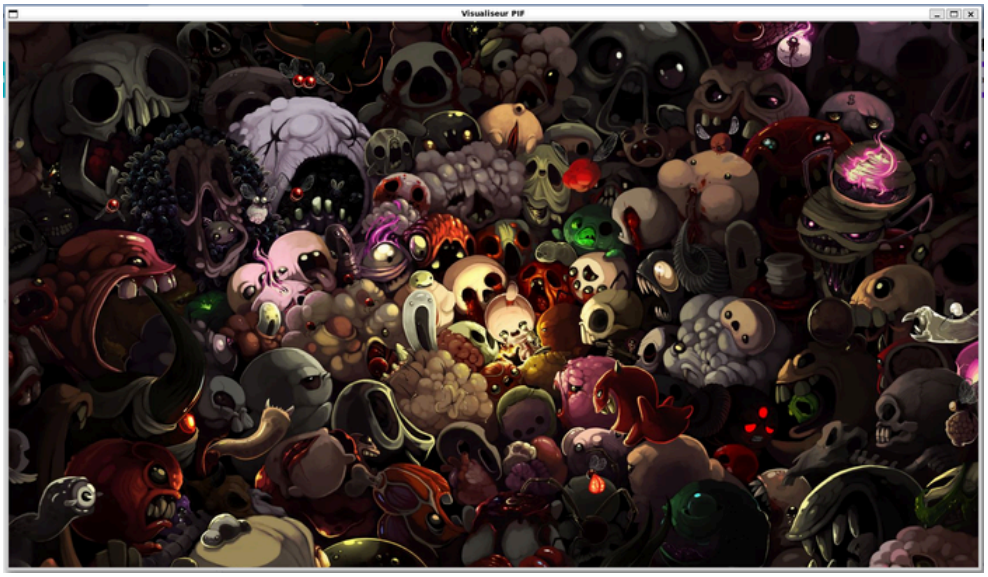
On aura besoin de lire cette partie bit par bit, car le nombre bits occupé par un pixel est variable. Cependant en java il n’est pas possible de lire un bit, c’est pour cela qu’on a créé la méthode “getBit”. Elle lit un octet puis elle utilise le décalage bit à bit et un ET binaire pour pouvoir isoler le bit en fonction d’un certain index. Une fois le bit isolé il est renvoyé par la méthode.

Pour la suite on va devoir récupérer les codes canonique du fichier et les comparer avec les notre stocké dans “HashMap” pour obtenir l’intensité de la composante RGB. La méthode “decoleValue” a été conçu pour cela. Elle va lire des bits et les stocker dans une variable, la méthode va tourner en boucle jusqu’a ce que la variable corresponde à un code canonique présent de le “HashMap”.

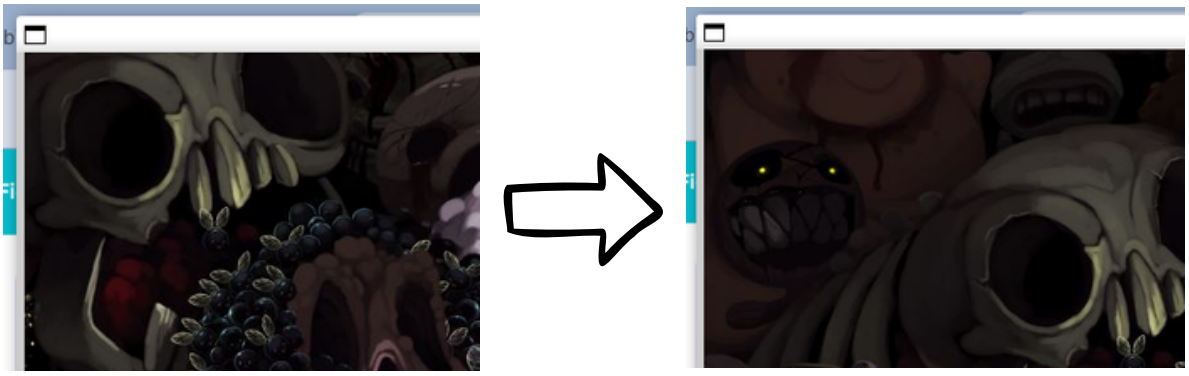
Pour créer l’image on va utiliser la méthode “buildImage”. Elle parcourt l'image pixel par pixel (largeur et hauteur) et appelle decodeValue successivement pour les trois composantes (Rouge, Vert, Bleu). Une fois les trois intensités récupérées, elle recrée la couleur finale et l'applique à la BufferedImage.

Présentation de l’exécution du visualisateur

Après exécution du programme et après avoir choisi l’image **.pif** (cela peut prendre du temps), l’image apparaitra dans une JFrame mais dans une fenêtre limité à 1200 px et 800 px.

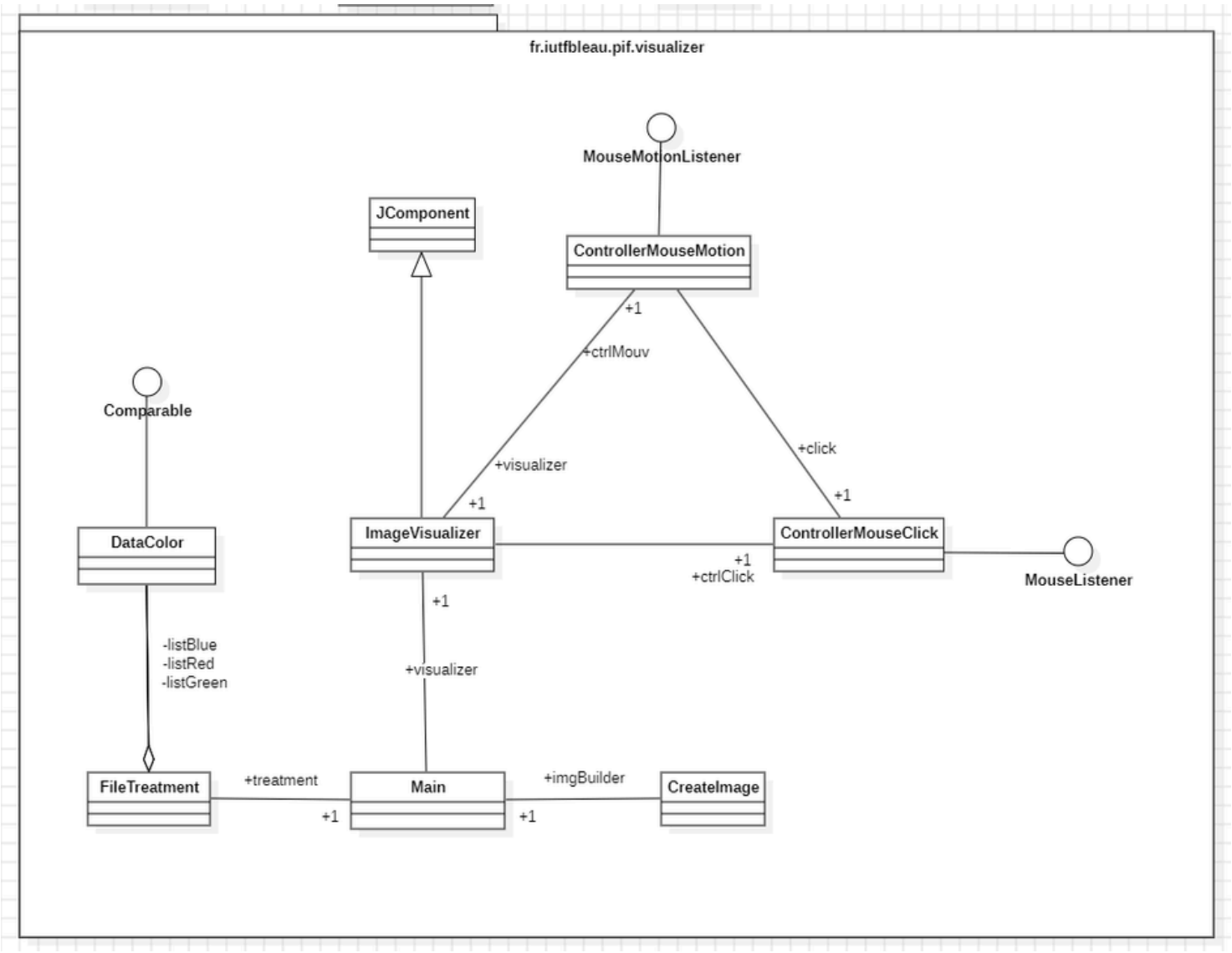


Il faut maintenir soit le clique gauche,droit ou la molette pour pouvoir se déplacer dans l’image à la **“drag and drop”**. Prenons en exemple le côté en haut à gauche de l’image.



puisque l’image est trop grande par rapport aux normes que l’on a imposé, on peut se déplacer et donc voir le “reste” de l’image. On voit ci-dessous que j’ai pu déplacer l’image.

Présentation de la structure du programme et diagramme de classes simplifié



Le programme du visualisateur se décompose en 3 grandes parties :

- La première partie contient les classes “FileTreatment” et “DataColor”. Cette partie s’occupe de la création des tables de codes et du traitement de l’en-tête du fichier qui contient les dimensions de l’image à afficher.
- La deuxième partie contient la classe “CreateImage”, elle s’occupe de créer l’image à partir des tables de codes de la première partie.
- La troisième partie contient les classes “ImageVisualizer”, “ControllerMouseClick” et “ControllerMouseMotion”, elle s’occupe de gérer la fonctionnalité de drag and drop

L’ensemble de ces 3 parties permettent au visualisateur de fonctionner correctement.

CONCLUSION

Michel

Ce projet ne m'a pas du tout plu, j'adore utiliser ma créativité durant les projets, mais malheureusement le projet ne m'en a pas permis. Il était énormément centré sur du travail backend. En temps normal j'ai une préférence sur le frontend mais j'éprouve tout de même du plaisir à réaliser du travail backend, cependant j'ai trouvé que le ratio entre les deux était trop déséquilibré. Ce projet m'a tout de même permit de m'améliorer sur les calculs bit à bit, de plus j'ai adoré l'expérience de lancer le visualisateur et de le voir fonctionner pour la première fois. J'ai lu le sujet de la prochaine SAE par curiosité, j'ai vraiment hâte de la commencer, le mode hardcore à définir soi-même est une super idée je trouve et j'ai réellement hâte de commencer cette SAE.

Emmanuel

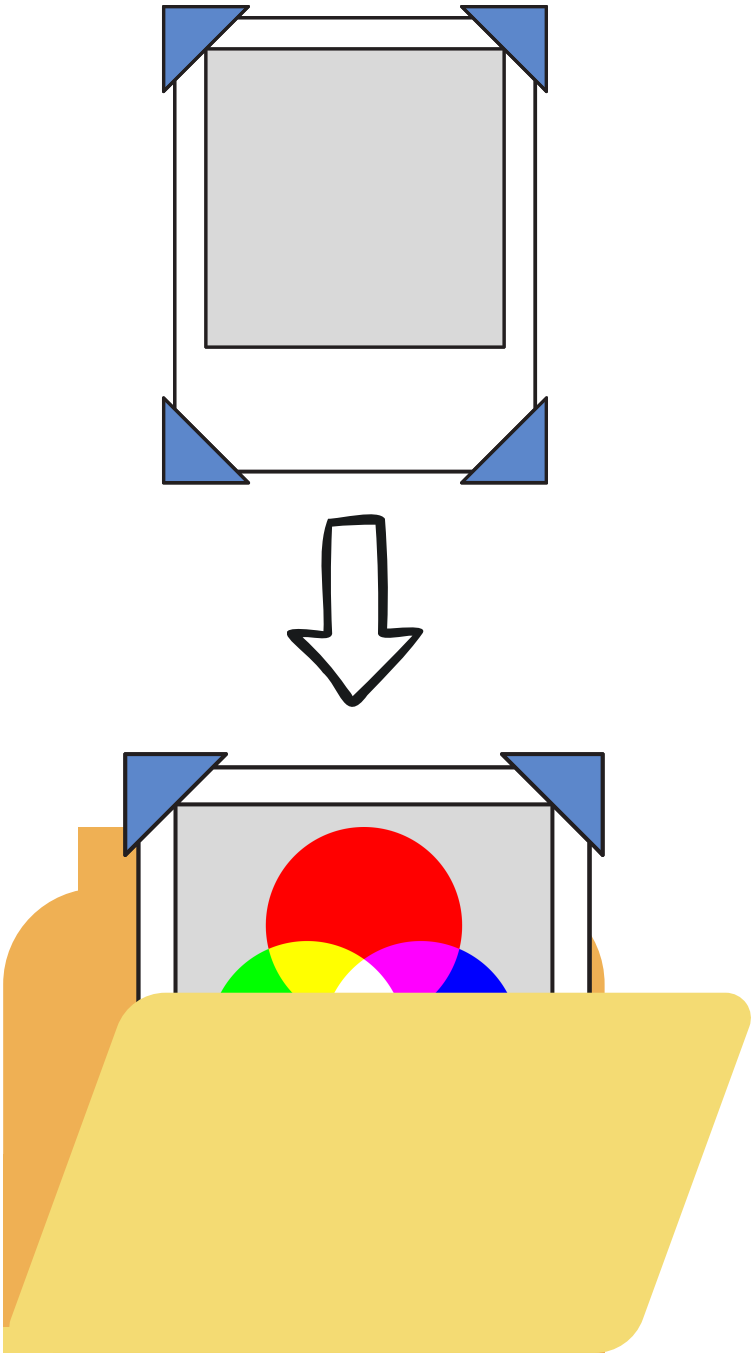
Personnellement, ce projet ne m'a pas vraiment plu, notamment car son lancement en pleine semaine de partiels, ses contraintes et sa complexité ont rendu le développement assez lourd et parfois frustrant. J'ai travaillé sur la logique d'affichage, l'implémentation de l'algorithme de Huffman et des codes canoniques, ce qui m'a permis de mieux comprendre la gestion des fichiers binaires à bas niveau. J'ai surtout hâte du prochain projet, de développer un jeu sur android et apprendre à utiliser de nouveaux outils, mais aussi car ça fait longtemps que je n'ai pas développé de jeux. Au final, même si ce projet n'a pas été le plus motivant, il m'a clairement appris à écrire mon code de façon plus méthodique et disciplinée. J'ai surtout hâte du prochain projet, de développer un jeu sur android et apprendre à utiliser de nouveaux outils, mais aussi car ça fait longtemps que je n'ai pas développé de jeux.

Rayan

Je ne sais même pas par où commencer, il y a beaucoup de choses à dire. Déjà, la gestion de projet était vraiment clé dans celui-là plus que dans les autres, car on avait une semaine de partiels en plus d'avoir un délai plus cours que les FI pour cette SAé, alors c'était extrêmement compliqué en termes d'organisation de trouver le temps nécessaire pour travailler.

Pour ma part, je me suis occupé de tout le convertisseur de A à Z, alors la charge de travail était assez gigantesque. Malgré ça, j'ai découvert des manières d'organiser les données que je n'aurais imaginées avant, et également l'enfer de l'optimisation afin de ne pas avoir un fichier qui se convertit en .pif en 2 minutes, alors elle a été très valorisante bien que stressante surtout sur la fin. Pour résumer cette SAé, elle n'était pas très compliquée à mettre en oeuvre mais vraiment dure à comprendre, la logique derrière le convertisseur est une chose nouvelle qu'on a découverte qui a été assez dure à appréhender, mais on a tout de même fini avec un projet dont je suis fier avec un convertisseur qui est à mon sens vraiment beau, et je veux dire par là que en relisant mon code, je me dis justement au fur et à mesure qu'il y a de bonnes idées que j'ai trouvées. En bref, le sujet a été vraiment intéressant en terme de back-end (que je maîtrise mieux que le front), mais avec quelques complications en terme de temps qui ne nous ont quand même pas empêché de rendre un travail satisfaisant.

SAE 3.2 FA - PRIMITIVE IMAGE FORMAT
2025 - 2026



Rayan BISSON, Michel TRAN &
Emmanuel SRIVASTAVA-TIAMZON